const with objects

Why you can still change the object

const.js

```
1  const user = { name: "Aman" };
2  user.name = "Raj"; // works
3  console.log(user.name); // "Raj"
4  user = { name: "Raj" }; // TypeError
```

output `"Raj"` `TypeError`

Why it happens

01 A reference
user holds the ADDRESS of the object in memory, not the object.

02 const locks the address
You cannot point user to a different object.

03 Inside is open
Properties can still be added, changed or deleted.

04 Same for arrays
const arr = [] still allows arr.push(1).

@specsycoder

Locking the object

Object.freeze and its limits

freeze.js

```
1  const user = Object.freeze({ name: "Aman" });
2  user.name = "Raj"; // ignored (TypeError in strict mode)
3  console.log(user.name); // "Aman"
4
5  const box = Object.freeze({ inner: { x: 1 } });
6  box.inner.x = 2; // works! freeze is shallow
```

method	add	change	delete
const only	yes	yes	yes
Object.preventExtensions	no	yes	yes
Object.seal	no	yes	no
Object.freeze	no	no	no

Traps to mention

- **freeze is shallow.** Nested objects stay changeable unless you freeze them too.
- **Silent in sloppy mode.** Changes to a frozen object fail without an error unless strict mode is on.
- **const is about the binding,** let vs const has nothing to do with object contents.

Your 20-second answer

"const locks the variable, not the value. An object variable holds a reference, so const stops me from pointing it to a new object, but I can still change its properties. To make the object itself read-only I use Object.freeze, which is shallow, so nested objects need freezing too."